

LAPORAN PRAKTIKUM STRUKTUR DATA

Pekan 7

“Algoritma Sorting”



OLEH:

LEXI MULIA YUNASPI

2511531006

**MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : DR WAHYUDI ,MT**

ASISTEN PRAKTIKUM : M.GHALID

**FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Puji syukur penulis ucapkan kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini dibuat untuk memenuhi tugas praktikum Struktur Data mengenai penerapan algoritma sorting menggunakan bahasa pemrograman Java, baik berbasis console maupun GUI. Pada praktikum ini penulis mempelajari beberapa algoritma pengurutan seperti Insertion Sort, Selection Sort, dan Bubble Sort serta implementasinya pada data array dan data mahasiswa menggunakan Java Swing.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan ini di masa mendatang. Semoga laporan praktikum ini dapat memberikan manfaat dan menambah pemahaman pembaca mengenai konsep sorting dan implementasi GUI pada Java.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Langkah Langkah Praktikum.....	3
2.2 Langkah Langkah Tugas Praktikum.....	30
BAB III PENUTUP.....	45
3.1 Kesimpulan.....	45
DAFTAR PUSTAKA.....	46

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi membuat proses pengolahan data menjadi semakin penting dalam dunia komputer. Salah satu proses dasar dalam pengolahan data adalah sorting atau pengurutan data. Pengurutan data digunakan untuk mempermudah pencarian, pengelompokan, dan pengolahan informasi sehingga data menjadi lebih terstruktur dan mudah dipahami. Dalam pemrograman, terdapat berbagai algoritma sorting yang memiliki cara kerja dan tingkat efisiensi yang berbeda. Pada praktikum ini digunakan bahasa pemrograman Java untuk mengimplementasikan algoritma Insertion Sort, Selection Sort, dan Bubble Sort. Selain menggunakan tampilan console, praktikum juga dikembangkan menggunakan Java Swing berbasis GUI agar aplikasi lebih interaktif dan mudah digunakan pengguna. Program GUI memungkinkan pengguna memasukkan data mahasiswa, melakukan sorting nama mahasiswa, serta melihat proses sorting secara langsung. Melalui praktikum ini mahasiswa dapat memahami cara kerja algoritma sorting secara teori maupun implementasi program. Selain itu, mahasiswa juga belajar mengenai penggunaan class, object, method, array, ArrayList, dan komponen GUI pada Java. Dengan demikian praktikum ini dapat meningkatkan kemampuan logika pemrograman dan pemahaman struktur data.

1.2 Tujuan

konsep dasar algoritma sorting pada struktur data.

1. Mengimplementasikan algoritma Insertion Sort, Selection Sort, dan Bubble Sort menggunakan Java.
2. Memahami penggunaan class dan object dalam pemrograman berorientasi objek.
3. Membuat aplikasi GUI sederhana menggunakan Java Swing.
4. Menampilkan proses pengurutan data mahasiswa berdasarkan nama secara interaktif.

1.3 Manfaat

Adapun manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mahasiswa dapat memahami konsep dasar algoritma sorting seperti Insertion Sort, Selection Sort, dan Bubble Sort pada bahasa pemrograman Java.
2. Mahasiswa dapat mengetahui proses pengurutan data secara bertahap menggunakan berbagai metode sorting yang berbeda.
3. Mahasiswa dapat melatih kemampuan dalam membuat program berbasis GUI menggunakan Java Swing sehingga program menjadi lebih interaktif dan mudah digunakan.
4. Mahasiswa dapat memahami penggunaan class, method, object, array, serta ArrayList dalam pemrograman berorientasi objek.
5. Mahasiswa dapat mengetahui cara menampilkan data mahasiswa ke dalam tabel menggunakan komponen JTable dan DefaultTableModel.

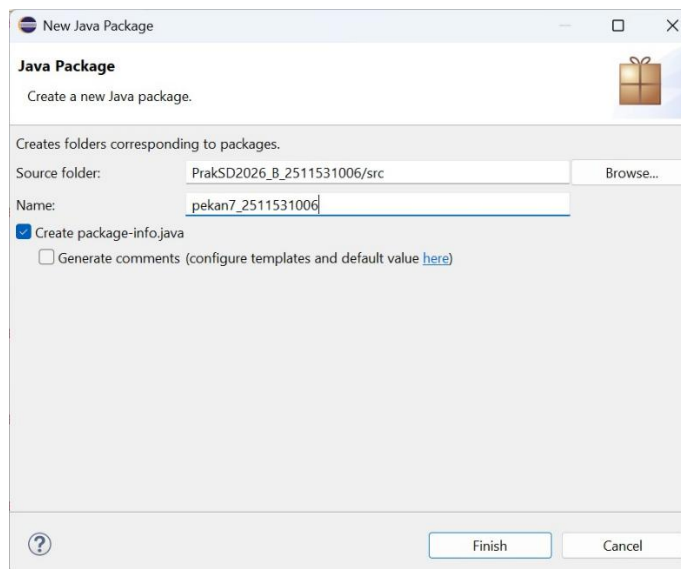
BAB II

PEMBAHASAN

2.1 Laporan Praktikum

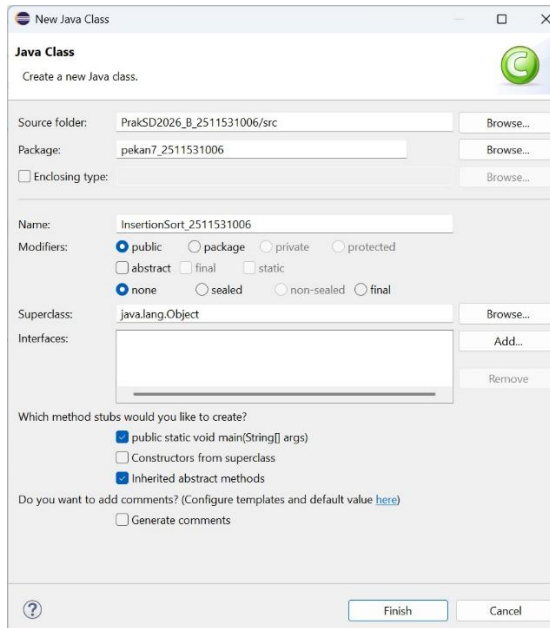
1. Program 1

a. Membuat Package Java



Pada tahap awal, dilakukan pembuatan package baru di Eclipse dengan nama pekan7_2511531006. Package digunakan untuk mengelompokkan file program agar lebih rapi dan mudah dikelola. Pembuatan package dilakukan melalui menu New → Package kemudian mengisi nama package sesuai format yang ditentukan. Pada tampilan “New Java Package”, source folder yang digunakan adalah PrakSD2026_B_2511531006/src. Setelah nama package diisi, klik tombol Finish untuk menyelesaikan proses pembuatan package. Dengan adanya package, file program akan tersusun dengan lebih teratur di dalam project Java.

b. Membuat Class Java



Setelah package selesai dibuat, langkah berikutnya adalah membuat class baru dengan nama InsertionSort_2511531006. Pembuatan class dilakukan melalui menu New → Class lalu memilih package pekan7_2511531006. Pada bagian name diisi dengan nama class sesuai algoritma yang digunakan. Opsi public static void main(String[] args) dicentang agar Eclipse otomatis membuat method main sebagai titik awal program dijalankan. Class ini digunakan sebagai tempat penulisan seluruh kode program insertion sort. Setelah semua pengaturan selesai, klik tombol Finish untuk membuat class baru

c. Package dan Class Program

```
1 package pekan7_2511531006;  
2  
3 public class InsertionSort_2511531006 {
```

Bagian ini menunjukkan bahwa program berada di dalam package pekan7_2511531006 dan menggunakan class bernama InsertionSort_2511531006. Package berfungsi untuk mengorganisasi file program, sedangkan class menjadi wadah utama seluruh kode Java. Nama class dibuat sesuai dengan algoritma sorting

yang digunakan agar mudah dikenali. Penambahan NIM pada nama class bertujuan untuk membedakan program dengan milik mahasiswa lain. Struktur awal seperti ini wajib ada dalam program Java. Tanpa class, program tidak dapat dijalankan.

d. Method Insertion Sort

```
4 public static void insertionSort_1006(int[] arr_1006) {
```

Bagian ini merupakan method yang digunakan untuk melakukan proses pengurutan data menggunakan algoritma insertion sort. Method menerima parameter berupa array integer arr_1006. Kata static digunakan agar method dapat dipanggil langsung tanpa membuat objek terlebih dahulu. Sedangkan void berarti method tidak mengembalikan nilai. Algoritma insertion sort bekerja dengan cara mengambil satu elemen lalu menyisipkannya ke posisi yang sesuai pada bagian array yang sudah terurut. Method ini menjadi inti utama dalam proses sorting data.

e. Menentukan Panjang Array

```
5 int n_1006 = arr_1006.length;
```

Kode ini digunakan untuk menyimpan jumlah elemen array ke dalam variabel n_1006. Fungsi .length digunakan untuk mengetahui panjang array secara otomatis. Pada program ini jumlah elemen array adalah 7 data. Variabel n_1006 nantinya digunakan sebagai batas perulangan pada proses sorting. Dengan menggunakan .length, program menjadi lebih fleksibel jika jumlah data berubah. Cara ini juga membuat penulisan kode menjadi lebih efisien dan rapi.

f. Perulangan Utama

```
6 for (int i_1006 = 1; i_1006 < n_1006; i_1006++) {
```


Perulangan for digunakan untuk menjalankan proses insertion sort dari indeks ke-1 sampai elemen terakhir array. Indeks dimulai dari 1 karena elemen pertama dianggap sudah berada pada posisi yang benar. Variabel `i_1006` digunakan untuk menandai data yang sedang diproses. Pada setiap iterasi, program akan mengambil satu data lalu membandingkannya dengan elemen sebelumnya. Jika posisi belum sesuai maka data akan digeser ke kanan. Proses ini terus dilakukan sampai semua data terurut.

g. Menyimpan Nilai Key

```
7      int key_1006 = arr_1006[i_1006];  
8      int j_1006 = i_1006 - 1;
```

Variabel `key_1006` digunakan untuk menyimpan nilai array yang sedang dibandingkan. Nilai ini nantinya akan dipindahkan ke posisi yang tepat setelah proses perbandingan selesai. Sedangkan variabel `j_1006` digunakan untuk memeriksa elemen sebelumnya dalam array. Nilai `j_1006` dimulai dari `i_1006 - 1` karena insertion sort selalu membandingkan data dengan bagian kiri array yang dianggap sudah terurut. Penggunaan variabel `key` membuat data asli tetap aman selama proses penggeseran berlangsung. Bagian ini merupakan inti penting dalam insertion sort.

h. Proses Perbandingan dan Pergeseran

```
9      while (j_1006 >= 0 && arr_1006[j_1006] > key_1006) {  
10         arr_1006[j_1006 + 1] = arr_1006[j_1006];  
11         j_1006--;  
12     }
```

Bagian `while` digunakan untuk membandingkan nilai `key` dengan elemen sebelumnya. Jika elemen sebelumnya lebih besar dari `key`, maka data tersebut akan digeser satu langkah ke kanan. Pergeseran dilakukan menggunakan perintah `arr_1006[j_1006 + 1] = arr_1006[j_1006]`. Setelah itu nilai `j_1006` dikurangi satu agar proses pengecekan berlanjut ke elemen sebelumnya lagi. Perulangan akan berhenti jika ditemukan

elemen yang lebih kecil dari key atau posisi sudah mencapai awal array. Dengan proses ini, data akan tersusun secara ascending.

i. Menempatkan Key pada Posisi yang Tepat

```
13         arr_1006[j_1006 + 1] = key_1006;  
14     }  
15 }
```

Kode ini digunakan untuk memasukkan nilai key ke posisi yang benar setelah proses penggeseran selesai. Posisi $j_1006 + 1$ merupakan tempat yang sesuai agar urutan data tetap teratur. Jika tidak dilakukan langkah ini, maka nilai key akan hilang karena sebelumnya hanya disimpan sementara. Setelah key ditempatkan, satu proses insertion selesai dilakukan. Langkah ini akan terus diulang sampai seluruh data array berhasil diurutkan. Hasil akhirnya adalah array tersusun dari nilai terkecil ke terbesar.

j. Method Main

```
16 public static void main(String[] args) {
```

Method main merupakan bagian utama program yang pertama kali dijalankan oleh Java. Semua proses dalam program dimulai dari method ini. Pada bagian ini dilakukan pembuatan array, pemanggilan method insertion sort, dan menampilkan hasil output ke layar. Method main sangat penting karena menjadi pusat eksekusi program. Tanpa method main, program Java tidak dapat dijalankan secara normal. Oleh karena itu, bagian ini selalu ada dalam program Java dasar.

k. Deklarasi Array

```
17 int arr_1006[] = {23, 70, 45, 8, 32, 56, 1};
```

Bagian ini digunakan untuk membuat array integer yang berisi beberapa data angka. Data awal masih dalam kondisi acak dan belum terurut. Nilai array yang digunakan adalah 23, 70, 45, 8, 32, 56, dan 1. Data inilah yang nantinya akan diproses menggunakan algoritma insertion sort. Penggunaan array memudahkan penyimpanan banyak data dalam satu variabel. Array juga sering digunakan dalam proses sorting dan searching pada struktur data.

1. Menampilkan Array Sebelum Sorting

```
19 System.out.print("array yang belum terurut:\n");
20 for (int i_1006 = 0; i_1006 < n_1006; i_1006++) {
21     System.out.print(arr_1006[i_1006] + " ");
22 }
```

Kode ini digunakan untuk menampilkan isi array sebelum dilakukan proses sorting. Program menggunakan perulangan for untuk mencetak seluruh elemen array satu per satu ke layar. Hasil yang ditampilkan menunjukkan data masih dalam kondisi acak. Tampilan ini penting agar pengguna dapat membandingkan keadaan array sebelum dan sesudah sorting dilakukan. Dengan demikian proses kerja insertion sort menjadi lebih mudah dipahami. Output awal array adalah 23 70 45 8 32 56 1.

m. Memanggil Method Sorting

```
24 insertionSort_1006(arr_1006);
```

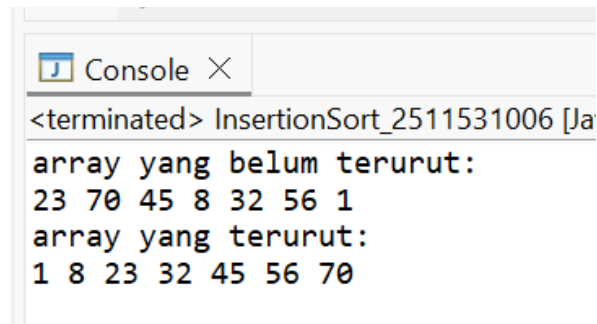
Baris ini digunakan untuk memanggil method insertion sort yang telah dibuat sebelumnya. Parameter yang dikirim adalah array arr_1006. Setelah method dipanggil, program mulai melakukan proses pengurutan data secara ascending. Semua elemen array akan dibandingkan dan dipindahkan ke posisi yang sesuai. Pemanggilan method membuat program lebih terstruktur dan mudah dibaca. Selain itu, penggunaan method juga memudahkan jika program ingin digunakan kembali di tempat lain.

n. Menampilkan Array Setelah Sorting

```
25 System.out.print("array yang terurut:\n");  
26 for (int i_1006 = 0; i_1006 < n_1006; i_1006++) {  
27     System.out.print(arr_1006[i_1006] + " ");  
28 }
```

Bagian ini digunakan untuk menampilkan array setelah proses sorting selesai dilakukan. Program kembali menggunakan perulangan for untuk mencetak seluruh isi array. Kali ini data sudah tersusun dari kecil ke besar karena telah diproses menggunakan insertion sort. Hasil akhirnya menjadi 1 8 23 32 45 56 70. Tampilan ini membuktikan bahwa algoritma insertion sort berhasil berjalan dengan baik. Dengan melihat hasil sebelum dan sesudah sorting, pengguna dapat memahami perubahan data secara jelas.

o. Hasil Output Program

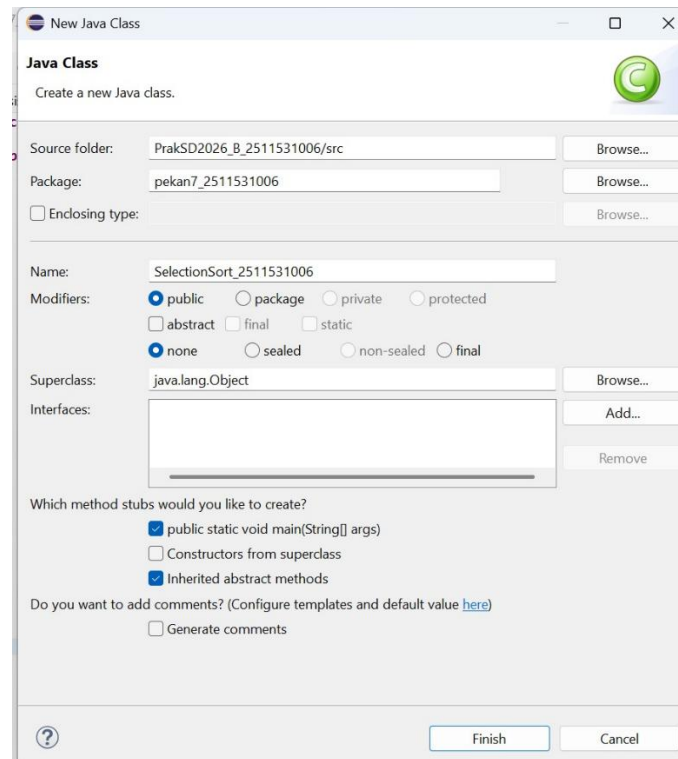


```
<terminated> InsertionSort_2511531006 [Ja  
array yang belum terurut:  
23 70 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 70
```

Output di atas menunjukkan hasil eksekusi program insertion sort yang telah dibuat. Pada bagian pertama ditampilkan array sebelum dilakukan proses sorting, yaitu 23 70 45 8 32 56 1. Data tersebut masih dalam kondisi acak dan belum tersusun secara berurutan. Setelah method `insertionSort_1006()` dijalankan, program berhasil mengurutkan data dari nilai terkecil ke terbesar. Hasil akhirnya menjadi 1 8 23 32 45 56 70 sehingga membuktikan bahwa algoritma insertion sort berjalan dengan baik.

2. Program 2

a. Membuat Class Java Selection Sort



Pada tahap ini dilakukan pembuatan class baru di Eclipse dengan nama SelectionSort_2511531006. Class dibuat melalui menu New → Class kemudian memilih package pekan7_2511531006. Nama class disesuaikan dengan algoritma yang digunakan yaitu Selection Sort serta ditambahkan NIM agar sesuai dengan format tugas praktikum. Pada bagian method stubs dipilih public static void main(String[] args) agar Eclipse otomatis membuat method main. Class ini digunakan sebagai tempat seluruh kode program selection sort. Setelah semua pengaturan selesai, klik tombol Finish untuk membuat class baru.

b. Package dan Class Program

```

1 package pekan7_2511531006;
2
3 public class SelectionSort_1006 {
4

```

Bagian ini menunjukkan bahwa program berada di dalam package pekan7_2511531006. Package digunakan untuk mengelompokkan file program agar lebih rapi dan mudah dikelola. Sedangkan class SelectionSort_1006 menjadi tempat utama penulisan program selection sort. Nama class dibuat sesuai algoritma yang digunakan supaya mudah dipahami. Penambahan angka 1006 digunakan sebagai identitas sesuai NIM mahasiswa. Struktur package dan class wajib ada dalam program Java agar program dapat dijalankan dengan baik.

c. Method Selection Sort

```

4
5 public static void selectionSort_1006(int[] arr_1006) {

```

Bagian ini merupakan method utama yang digunakan untuk melakukan proses pengurutan data menggunakan algoritma selection sort. Method menerima parameter berupa array integer arr_1006. Kata static digunakan agar method dapat dipanggil langsung tanpa membuat objek terlebih dahulu. Sedangkan void berarti method tidak mengembalikan nilai. Algoritma selection sort bekerja dengan cara mencari nilai terkecil dari array lalu menukarnya dengan posisi awal. Proses ini dilakukan berulang sampai seluruh data tersusun secara terurut.

d. Menentukan Panjang Array

```

6 int n_1006 = arr_1006.length;

```

Kode ini digunakan untuk menyimpan jumlah elemen array ke dalam variabel n_1006. Fungsi .length digunakan untuk mengetahui panjang array secara otomatis.

Pada program ini jumlah data array adalah 7 elemen. Variabel `n_1006` nantinya digunakan sebagai batas perulangan dalam proses sorting. Penggunaan `.length` membuat program lebih fleksibel jika jumlah data berubah. Cara ini juga membuat kode menjadi lebih singkat dan efisien.

e. Perulangan Utama Selection Sort

```
70         for (int i_1006 = 0; i_1006 < n_1006; i_1006++) {
```

Perulangan `for` digunakan untuk menentukan posisi data yang akan diisi dengan nilai terkecil. Variabel `i_1006` berfungsi sebagai indeks utama dalam proses sorting. Pada setiap iterasi, program akan mencari elemen terkecil dari bagian array yang belum terurut. Setelah nilai terkecil ditemukan, data akan ditukar dengan posisi indeks `i_1006`. Proses ini dilakukan terus menerus hingga seluruh elemen array tersusun dari kecil ke besar. Bagian ini menjadi proses utama dalam algoritma selection sort.

f. Menentukan Indeks Minimum

```
8         int minIndex_1006 = i_1006;
```

Variabel `minIndex_1006` digunakan untuk menyimpan indeks dari nilai terkecil pada array. Awalnya nilai minimum dianggap berada pada indeks `i_1006`. Setelah itu program akan melakukan pengecekan ke elemen berikutnya untuk mencari nilai yang lebih kecil. Jika ditemukan nilai yang lebih kecil, maka `minIndex_1006` akan diperbarui. Variabel ini sangat penting karena digunakan untuk menentukan data mana yang akan ditukar. Dengan cara ini proses selection sort dapat berjalan dengan benar.

g. Perulangan Pencarian Nilai Terkecil

```
90         for (int j_1006 = i_1006 + 1; j_1006 < n_1006; j_1006++) {
```

Perulangan ini digunakan untuk mencari nilai terkecil pada bagian array yang belum terurut. Variabel `j_1006` dimulai dari `i_1006 + 1` karena elemen sebelumnya sudah dianggap terurut. Program akan membandingkan setiap elemen dengan nilai minimum sementara. Jika ditemukan nilai yang lebih kecil, maka indeks minimum akan diperbarui. Perulangan ini terus berjalan sampai akhir array. Dengan demikian program dapat menemukan elemen terkecil pada setiap iterasi.

h. Proses Perbandingan Data

```
10     if (arr_1006[j_1006] < arr_1006[minIndex_1006]) {  
11         minIndex_1006 = j_1006;  
12     }  
13 }  
14
```

Bagian `if` digunakan untuk membandingkan nilai array pada indeks `j_1006` dengan nilai minimum sementara. Jika nilai array lebih kecil, maka `minIndex_1006` akan diubah sesuai indeks baru tersebut. Dengan cara ini program dapat mengetahui posisi nilai terkecil dalam array. Proses perbandingan dilakukan terus menerus selama perulangan berjalan. Setelah selesai, variabel `minIndex_1006` akan berisi posisi elemen terkecil. Langkah ini merupakan inti utama dari algoritma selection sort.

i. Proses Pertukaran Data

```
14  
15     int temp_1006 = arr_1006[i_1006];  
16     arr_1006[i_1006] = arr_1006[minIndex_1006];  
17     arr_1006[minIndex_1006] = temp_1006;  
18 }  
19
```

Kode ini digunakan untuk menukar posisi data. Variabel `temp_1006` digunakan sebagai penyimpanan sementara agar data tidak hilang saat proses pertukaran dilakukan. Nilai pada indeks `i_1006` ditukar dengan nilai terkecil yang berada pada `minIndex_1006`. Setelah pertukaran selesai, posisi awal array akan berisi nilai

terkecil. Proses ini dilakukan berulang pada setiap iterasi hingga semua data tersusun rapi. Teknik pertukaran seperti ini sering digunakan dalam algoritma sorting.

j. Method Main

```
20 public static void main(String[] args) {
```

Method main merupakan bagian utama program yang pertama kali dijalankan oleh Java. Semua proses dalam program dimulai dari method ini. Pada bagian ini dilakukan deklarasi array, pemanggilan method selection sort, dan menampilkan hasil output ke layar. Method main menjadi pusat eksekusi program. Tanpa method main, program Java tidak dapat berjalan dengan normal. Oleh karena itu bagian ini selalu ada dalam program Java dasar.

k. Deklarasi Array

```
21 int arr_1006[] = {23, 78, 45, 8, 32, 56, 1};  
22
```

Bagian ini digunakan untuk membuat array integer yang berisi beberapa angka acak. Data awal array belum terurut karena nilai masih tersusun secara random. Nilai yang digunakan yaitu 23, 78, 45, 8, 32, 56, dan 1. Array ini nantinya akan diproses menggunakan algoritma selection sort. Penggunaan array memudahkan penyimpanan banyak data dalam satu variabel. Data inilah yang akan dibandingkan dan ditukar selama proses sorting berlangsung.

l. Menampilkan Array Sebelum Sorting

```

24         System.out.print("array yang belum terurut:\n");
25         for (int i_1006 = 0; i_1006 < n_1006; i_1006++) {
26
27             System.out.print(arr_1006[i_1006] + " ");
28         }
29

```

Kode ini digunakan untuk menampilkan isi array sebelum dilakukan proses sorting. Program menggunakan perulangan for untuk mencetak seluruh elemen array satu per satu. Hasil yang ditampilkan menunjukkan data masih dalam kondisi belum terurut. Tampilan ini penting agar pengguna dapat melihat perbedaan sebelum dan sesudah proses sorting dilakukan. Dengan demikian hasil kerja algoritma selection sort menjadi lebih jelas. Output awal array adalah 23 78 45 8 32 56 1.

m. Memanggil Method Sorting

```

31         selectionSort_1006(arr_1006);

```

Baris ini digunakan untuk memanggil method selection sort yang telah dibuat sebelumnya. Parameter yang dikirim adalah array arr_1006. Setelah method dipanggil, program akan mulai melakukan proses pengurutan data secara ascending. Semua elemen array akan dibandingkan dan ditukar sesuai posisi yang benar. Penggunaan method membuat program lebih terstruktur dan mudah dipahami. Selain itu, method juga mempermudah penggunaan ulang program di bagian lain.

n. Menampilkan Array Setelah Sorting

```

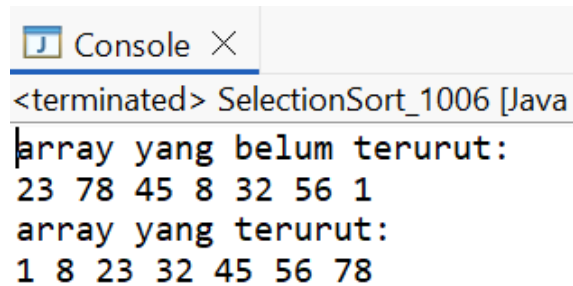
32         System.out.print("array yang terurut:\n");
33         for (int i_1006 = 0; i_1006 < n_1006; i_1006++)
34             System.out.print(arr_1006[i_1006] + " ");
35         }

```

Bagian ini digunakan untuk menampilkan array setelah proses sorting selesai dilakukan. Program kembali menggunakan perulangan for untuk mencetak isi array satu per satu. Kali ini data sudah tersusun dari kecil ke besar karena telah diproses menggunakan selection sort. Hasil akhirnya menjadi 1 8 23 32 45 56 78. Tampilan ini membuktikan bahwa algoritma selection sort berhasil berjalan dengan baik.

Dengan membandingkan hasil sebelum dan sesudah sorting, pengguna dapat memahami proses pengurutan data dengan lebih mudah.

o. Hasil Output Program

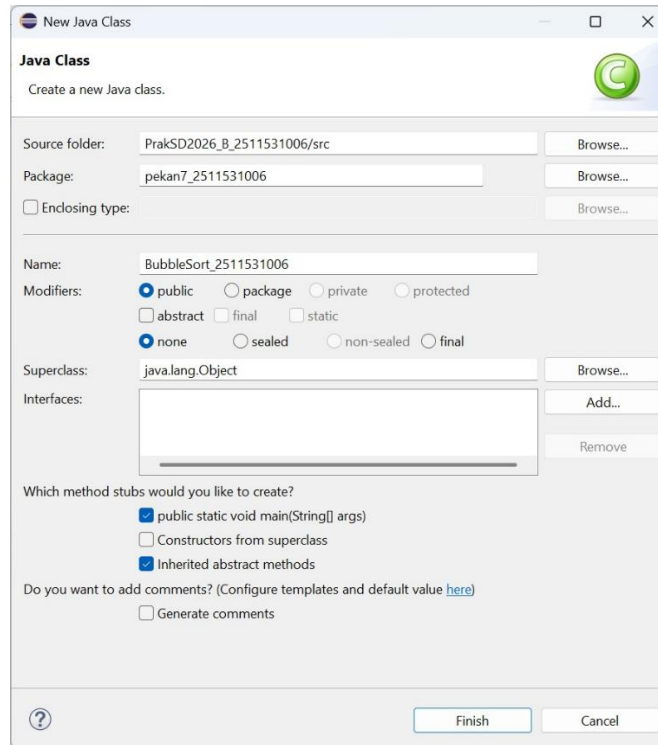


```
Console X
<terminated> SelectionSort_1006 [Java]
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Output di atas menunjukkan hasil eksekusi program Selection Sort yang telah dibuat. Pada bagian pertama ditampilkan array sebelum dilakukan proses sorting yaitu 23 78 45 8 32 56 1. Data tersebut masih dalam kondisi acak dan belum tersusun secara berurutan. Setelah method `selectionSort_1006()` dijalankan, program berhasil mengurutkan data dari nilai terkecil ke terbesar. Hasil akhirnya menjadi 1 8 23 32 45 56 78 sehingga membuktikan bahwa algoritma selection sort berjalan dengan baik dan benar.

3. Program 3

a. Membuat Class Java Bubble Sort



Pada tahap ini dilakukan pembuatan class baru di Eclipse dengan nama BubbleSort_2511531006. Class dibuat melalui menu New → Class kemudian memilih package pekan7_2511531006. Nama class disesuaikan dengan algoritma yang digunakan yaitu Bubble Sort serta ditambahkan NIM agar sesuai dengan format tugas praktikum. Pada bagian method stubs dipilih public static void main(String[] args) agar Eclipse otomatis membuat method main. Class ini nantinya digunakan sebagai tempat seluruh kode program bubble sort. Setelah semua pengaturan selesai, klik tombol Finish untuk membuat class baru.

b. Package dan Class Program

```
1 package pekan7_2511531006;  
2  
3 public class BubbleSort_1006{
```

Bagian ini menunjukkan bahwa program berada di dalam package pekan7_2511531006. Package digunakan untuk mengelompokkan file program agar lebih rapi dan mudah dikelola. Sedangkan class BubbleSort_1006 digunakan sebagai

tempat utama penulisan kode program bubble sort. Nama class dibuat sesuai dengan algoritma sorting yang digunakan sehingga mudah dipahami. Penambahan angka 1006 digunakan sebagai identitas sesuai NIM mahasiswa. Struktur package dan class merupakan bagian dasar yang wajib ada pada program Java.

c. Method Bubble Sort

```
40 public static void bubbleSort_1006(int[] arr_1006){
```

Bagian ini merupakan method utama yang digunakan untuk melakukan proses pengurutan data menggunakan algoritma bubble sort. Method menerima parameter berupa array integer arr_1006. Kata static digunakan agar method dapat dipanggil langsung tanpa membuat objek terlebih dahulu. Sedangkan void berarti method tidak mengembalikan nilai. Algoritma bubble sort bekerja dengan cara membandingkan dua elemen yang berdekatan lalu menukarnya jika posisi belum sesuai. Proses ini dilakukan berulang sampai seluruh data tersusun secara terurut.

d. Menentukan Panjang Array

```
5 int n_1006=arr_1006.length;
```

Kode ini digunakan untuk menyimpan jumlah elemen array ke dalam variabel n_1006. Fungsi .length digunakan untuk mengetahui panjang array secara otomatis. Pada program ini jumlah data array adalah 7 elemen. Variabel n_1006 digunakan sebagai batas perulangan dalam proses sorting. Dengan menggunakan .length, program menjadi lebih fleksibel jika jumlah data berubah. Cara ini juga membuat kode menjadi lebih singkat dan efisien.

e. Perulangan Pertama Bubble Sort

```
60 for(int i_1006=0;i_1006<n_1006;i_1006++){
```

Perulangan pertama digunakan untuk menentukan jumlah tahap proses bubble sort. Variabel i_1006 berfungsi sebagai penghitung banyaknya proses pengulangan

sorting. Pada setiap tahap, elemen terbesar akan berpindah ke bagian akhir array. Perulangan ini terus berjalan sampai seluruh data tersusun secara ascending. Semakin banyak jumlah data, maka semakin banyak pula proses perulangannya. Bagian ini menjadi proses utama dalam algoritma bubble sort.

f. Perulangan Kedua Bubble Sort

```
70 for(int j_1006=0;j_1006<n_1006-1;j_1006++){
```

Perulangan kedua digunakan untuk membandingkan elemen array yang saling berdekatan. Variabel j_1006 digunakan sebagai indeks pembanding antar data. Program akan mengecek apakah elemen kiri lebih besar dibanding elemen kanan. Jika kondisi benar, maka kedua elemen tersebut akan ditukar posisinya. Proses ini dilakukan berulang dari awal hingga akhir array. Dengan cara ini data yang lebih besar akan bergerak ke bagian belakang array seperti gelembung air sehingga disebut bubble sort.

g. Proses Perbandingan Data

```
80 if(arr_1006[j_1006]>arr_1006[j_1006+1]){
```

Bagian if digunakan untuk membandingkan dua elemen yang berdekatan dalam array. Jika elemen kiri lebih besar daripada elemen kanan, maka posisi kedua data harus ditukar. Kondisi ini digunakan agar data dapat tersusun dari kecil ke besar. Jika kondisi bernilai false maka data tidak akan ditukar. Proses perbandingan dilakukan terus menerus selama perulangan berjalan. Langkah ini merupakan inti utama dalam algoritma bubble sort.

h. Proses Pertukaran Data

```
9 int temp_1006=arr_1006[j_1006];  
10 arr_1006[j_1006]=arr_1006[j_1006+1];  
11 arr_1006[j_1006+1]=temp_1006;
```

Kode ini digunakan untuk menukar posisi dua data yang tidak sesuai urutannya. Variabel `temp_1006` digunakan sebagai tempat penyimpanan sementara agar data tidak hilang saat proses pertukaran berlangsung. Nilai pada indeks `j_1006` dipindahkan ke variabel sementara, kemudian nilai sebelah kanan dipindahkan ke kiri. Setelah itu nilai sementara dimasukkan kembali ke posisi kanan. Teknik seperti ini sering digunakan dalam proses sorting. Dengan proses pertukaran ini, data secara perlahan akan tersusun secara ascending.

i. Method Main

```
17  
18 public static void main(String[] args){
```

Method main merupakan bagian utama program yang pertama kali dijalankan oleh Java. Semua proses dalam program dimulai dari method ini. Pada bagian ini dilakukan deklarasi array, pemanggilan method bubble sort, dan menampilkan hasil output ke layar. Method main menjadi pusat eksekusi program. Tanpa method main, program Java tidak dapat berjalan dengan normal. Oleh karena itu bagian ini selalu ada pada program Java dasar.

j. Deklarasi Array

```
19 int arr_1006[]={23,78,45,8,32,56,1};
```

Bagian ini digunakan untuk membuat array integer yang berisi beberapa angka acak. Data awal array belum terurut karena nilainya masih tersusun secara random. Nilai array yang digunakan yaitu 23, 78, 45, 8, 32, 56, dan 1. Data inilah yang nantinya akan diproses menggunakan algoritma bubble sort. Penggunaan array memudahkan penyimpanan banyak data dalam satu variabel. Semua data akan dibandingkan dan ditukar selama proses sorting berlangsung.

k. Menampilkan Array Sebelum Sorting

```

21      System.out.print("array yang belum terurut:");
22      for(int i_1006=0;i_1006<n_1006;i_1006++){
23          System.out.print(arr_1006[i_1006]+" ");
24      }

```

Kode ini digunakan untuk menampilkan isi array sebelum dilakukan proses sorting. Program menggunakan perulangan for untuk mencetak seluruh elemen array satu per satu ke layar. Hasil yang ditampilkan menunjukkan bahwa data masih dalam kondisi belum terurut. Tampilan ini penting agar pengguna dapat membandingkan keadaan array sebelum dan sesudah sorting dilakukan. Dengan demikian proses kerja bubble sort dapat dipahami dengan lebih jelas. Output awal array adalah 23 78 45 8 32 56 1.

1. Memanggil Method Sorting

```

26      bubbleSort_1006(arr_1006);

```

Baris ini digunakan untuk memanggil method bubble sort yang telah dibuat sebelumnya. Parameter yang dikirim adalah array arr_1006. Setelah method dipanggil, program akan mulai melakukan proses pengurutan data secara ascending. Semua elemen array akan dibandingkan dan ditukar jika urutannya belum sesuai. Penggunaan method membuat program menjadi lebih terstruktur dan mudah dipahami. Selain itu method juga mempermudah penggunaan ulang kode program.

m. Menampilkan Array Setelah Sorting

```

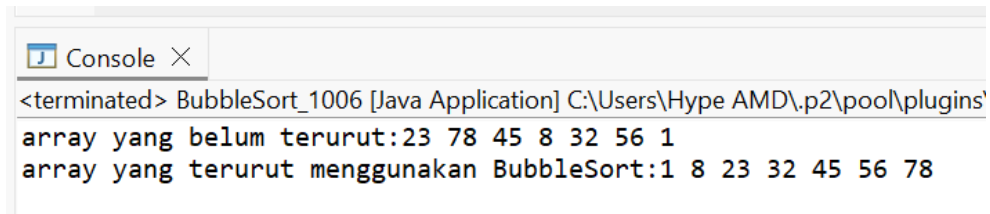
27      System.out.print("array yang terurut menggunakan BubbleSort:");
28      for(int i_1006=0;i_1006<n_1006;i_1006++){
29          System.out.print(arr_1006[i_1006]+" ");
30      }

```

Bagian ini digunakan untuk menampilkan array setelah proses sorting selesai dilakukan. Program kembali menggunakan perulangan for untuk mencetak isi array satu per satu. Kali ini data sudah tersusun dari kecil ke besar karena telah diproses menggunakan bubble sort. Hasil akhirnya menjadi 1 8 23 32 45 56 78. Tampilan ini membuktikan bahwa algoritma bubble sort berhasil berjalan dengan baik. Dengan

membandingkan hasil sebelum dan sesudah sorting, pengguna dapat memahami perubahan data secara jelas.

n. Hasil Output Program

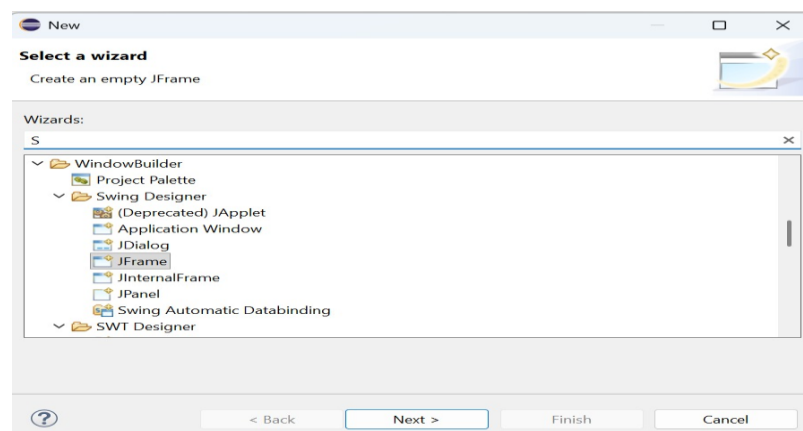


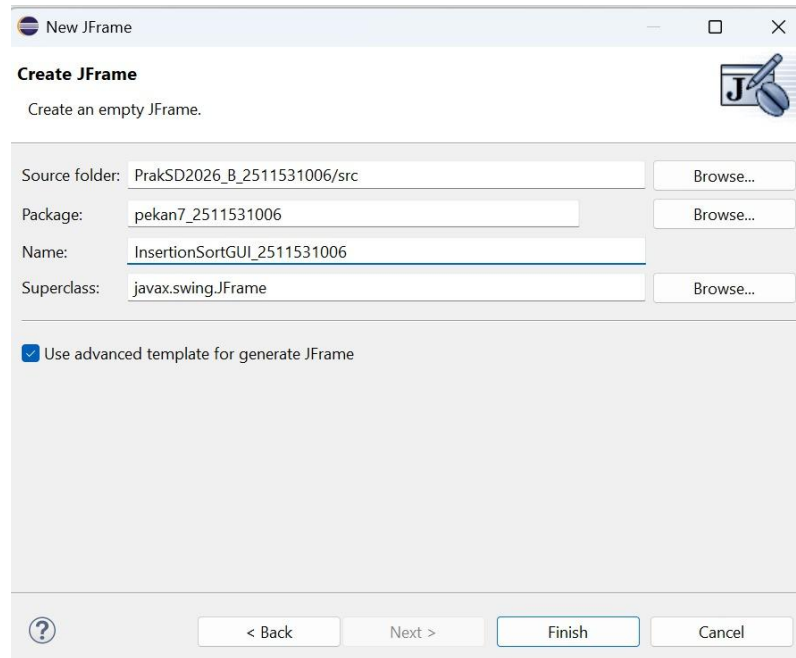
```
<terminated> BubbleSort_1006 [Java Application] C:\Users\Hype AMD\p2\pool\plugins'  
array yang belum terurut:23 78 45 8 32 56 1  
array yang terurut menggunakan BubbleSort:1 8 23 32 45 56 78
```

Output di atas menunjukkan hasil eksekusi program Bubble Sort yang telah dibuat. Pada bagian pertama, program menampilkan array sebelum dilakukan proses pengurutan yaitu 23 78 45 8 32 56 1. Data tersebut masih dalam kondisi acak dan belum tersusun secara ascending. Setelah method bubbleSort_1006() dijalankan, program mulai membandingkan setiap elemen yang berdekatan lalu menukarnya apabila posisi data belum sesuai. Proses ini dilakukan berulang sampai seluruh data tersusun dari nilai terkecil ke terbesar.

4. Program 4

a. Membuat JFrame Baru





```
20  
21 public class InsertionSortGUI_1006 extends JFrame {
```

public class InsertionSortGUI_1006 extends JFrame {.Pada tahap ini dibuat sebuah class GUI dengan nama InsertionSortGUI_1006 yang merupakan turunan dari JFrame. JFrame digunakan untuk membuat tampilan window atau jendela pada aplikasi Java Swing. Program ini dibuat untuk menampilkan proses insertion sort secara visual langkah demi langkah. Dengan menggunakan GUI, pengguna dapat melihat perubahan data secara langsung saat proses sorting berjalan. Nama class disesuaikan dengan algoritma yang digunakan serta ditambahkan NIM mahasiswa. Pembuatan JFrame dilakukan melalui menu Swing Designer → JFrame pada Eclipse.

b. Import Library Java Swing

```
2  
3+ import java.awt.BorderLayout; [..]  
--
```

Bagian import digunakan untuk memanggil library yang dibutuhkan dalam program GUI. Library javax.swing digunakan untuk membuat komponen tampilan seperti button, label, text field, dan panel. Sedangkan library java.awt digunakan untuk

mengatur layout, warna, ukuran, dan font pada tampilan aplikasi. Tanpa import library tersebut, komponen GUI tidak dapat dijalankan. Penggunaan Swing membuat tampilan program menjadi lebih interaktif dibanding program console biasa. Library ini sangat sering digunakan dalam pembuatan aplikasi desktop Java.

c. Deklarasi Variabel Program

```
23     private int[] array_1006;  
24     private JLabel[] labelArray_1006;  
25     private JButton stepButton_1006, resetButton_1006, setButton_1006;  
26     private JTextField inputField_1006;  
27     private JPanel panelArray_1006;  
28     private JTextArea stepArea_1006;
```

Bagian ini digunakan untuk mendeklarasikan variabel yang akan digunakan dalam program GUI insertion sort. Variabel array_1006 digunakan untuk menyimpan data angka yang akan diurutkan. JLabel digunakan untuk menampilkan isi array secara visual di layar. JButton digunakan untuk tombol aksi seperti set array, langkah berikutnya, dan reset. JTextField digunakan sebagai tempat pengguna memasukkan angka. Sedangkan JTextArea digunakan untuk menampilkan langkah-langkah proses sorting.

d. Constructor JFrame

```
32  
33     public InsertionSortGUI_1006() {  
34
```

Constructor digunakan untuk mengatur seluruh tampilan awal JFrame. Pada bagian ini dilakukan pengaturan judul window, ukuran frame, layout, serta penempatan komponen GUI. Constructor akan otomatis dijalankan ketika objek GUI dibuat. Semua tombol, panel, dan area teks juga diinisialisasi di dalam constructor ini. Dengan adanya constructor, tampilan program menjadi lebih terstruktur dan rapi. Bagian ini merupakan pusat pengaturan tampilan aplikasi GUI.

e. Mengatur Judul dan Ukuran JFrame

```

35     setTitle("Insertion Sort Langkah per Langkah");
36     setSize(750, 400);
37     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38     setLocationRelativeTo(null);
39     setLayout(new BorderLayout());

```

Kode ini digunakan untuk mengatur tampilan utama JFrame. Method setTitle() digunakan untuk memberi judul pada window aplikasi. setSize() digunakan untuk menentukan ukuran window dengan lebar 750 dan tinggi 400 pixel. setDefaultCloseOperation() digunakan agar program otomatis berhenti saat tombol close ditekan. Sedangkan setLocationRelativeTo(null) digunakan agar window muncul di tengah layar. Pengaturan ini membuat tampilan aplikasi menjadi lebih nyaman digunakan.

f. Membuat Input Angka

```

43     inputField_1006 = new JTextField(30);
44     setButton_1006 = new JButton("Set Array");

```

Bagian ini digunakan untuk membuat tempat input angka dan tombol set array. JTextField digunakan agar pengguna dapat memasukkan angka yang dipisahkan dengan tanda koma. Angka yang dimasukkan nantinya akan diproses menjadi array integer. Tombol Set Array digunakan untuk memulai pembentukan array dari input pengguna. Dengan adanya input field, program menjadi lebih fleksibel karena data dapat diubah sesuai kebutuhan. Komponen ini merupakan bagian penting dalam interaksi pengguna dengan program.

g. Membuat Tombol Kontrol

```

55     stepButton_1006 = new JButton("Langkah Selanjutnya");
56     resetButton_1006 = new JButton("Reset");

```

Kode ini digunakan untuk membuat tombol kontrol pada aplikasi. Tombol Langkah Selanjutnya digunakan untuk menjalankan proses insertion sort satu langkah demi satu langkah. Sedangkan tombol Reset digunakan untuk menghapus data dan mengembalikan tampilan ke kondisi awal. Penggunaan tombol membuat pengguna

dapat mengontrol jalannya proses sorting secara manual. Hal ini membantu pengguna memahami bagaimana insertion sort bekerja secara bertahap. GUI menjadi lebih interaktif dengan adanya tombol kontrol ini.

h. Membuat Area Log Langkah Sorting

```
62         stepArea_1006 = new JTextArea(8, 60);  
63         stepArea_1006.setEditable(false);
```

Bagian ini digunakan untuk membuat area teks yang menampilkan langkah-langkah sorting. JTextArea berfungsi sebagai tempat munculnya informasi proses insertion sort. Method setEditable(false) digunakan agar pengguna tidak dapat mengetik pada area tersebut. Semua langkah sorting akan ditampilkan secara otomatis oleh program. Dengan adanya area log, pengguna dapat melihat proses insertion sort secara detail. Hal ini sangat membantu dalam memahami algoritma insertion sort.

i. Event Tombol Set Array

```
--  
73         // Event Set Array  
74         setButton_1006.addActionListener(e -> setArrayFromInput());  
--
```

Kode ini digunakan untuk memberikan aksi pada tombol Set Array. Ketika tombol ditekan, program akan menjalankan method setArrayFromInput(). Method tersebut berfungsi membaca input angka dari pengguna lalu mengubahnya menjadi array integer. Setelah array berhasil dibuat, data akan ditampilkan pada panel visual. Penggunaan ActionListener membuat tombol dapat menjalankan fungsi tertentu saat diklik. Teknik ini merupakan dasar event handling pada Java Swing.

j. Method Membaca Input Array

```
83  
84         private void setArrayFromInput() {
```

Method ini digunakan untuk membaca data angka yang dimasukkan pengguna melalui JTextField. Program akan memisahkan angka berdasarkan tanda koma

menggunakan fungsi `split(",")`. Setelah itu setiap data diubah menjadi integer menggunakan `Integer.parseInt()`. Jika pengguna memasukkan selain angka, program akan menampilkan pesan error menggunakan `JOptionPane`. Method ini juga digunakan untuk menginisialisasi ulang proses sorting dari awal. Dengan method ini array dapat dibuat secara dinamis sesuai input pengguna.

k. Menampilkan Array ke GUI

```
105 labelArray_1006[k_1006] = new JLabel(String.valueOf(array_1006[k_1006]));
```

Bagian ini digunakan untuk menampilkan isi array ke dalam bentuk label pada GUI. Setiap elemen array akan dibuat menjadi satu `JLabel`. Label tersebut nantinya disusun secara horizontal pada panel array. Isi label akan berubah mengikuti proses insertion sort. Dengan tampilan visual ini pengguna dapat melihat perpindahan data secara langsung. Hal tersebut membuat proses pembelajaran sorting menjadi lebih mudah dipahami.

l. Method Proses Insertion Sort

```
116  
117 private void performStep() {
```

Method `performStep()` digunakan untuk menjalankan insertion sort secara bertahap. Setiap kali tombol langkah selanjutnya ditekan, program akan menjalankan satu proses insertion sort. Data akan dibandingkan lalu dipindahkan ke posisi yang sesuai. Setelah proses selesai, tampilan label array akan diperbarui. Method ini memungkinkan pengguna memahami insertion sort langkah demi langkah. Teknik ini sangat cocok digunakan untuk media pembelajaran algoritma sorting.

m. Proses Perbandingan dan Perpindahan Data

```
125  
126 while (j_1006 >= 0 && array_1006[j_1006] > key_1006) {  
127     array_1006[j_1006 + 1] = array_1006[j_1006];  
128     j_1006--;  
129  
130 }
```

n. Update Tampilan Label

```
147 private void updateLabels_1006() {
```

Method ini digunakan untuk memperbarui tampilan label setelah data berubah. Setiap elemen array akan ditampilkan ulang sesuai urutan terbaru. Tanpa method ini tampilan GUI tidak akan berubah meskipun data sudah diurutkan. Update label dilakukan setiap selesai satu langkah sorting. Dengan demikian pengguna dapat melihat perubahan data secara real-time. Hal ini membuat tampilan program menjadi lebih dinamis dan interaktif.

o. Method Reset

```
152  
153 private void reset() {
```

Method reset digunakan untuk menghapus seluruh data dan mengembalikan program ke kondisi awal. Input field akan dikosongkan dan panel array dibersihkan. Area log langkah sorting juga akan dihapus. Tombol langkah selanjutnya akan dinonaktifkan kembali sampai data baru dimasukkan. Fungsi reset memudahkan pengguna jika ingin mencoba data lain. Dengan adanya reset, program dapat digunakan berulang kali tanpa perlu menjalankan ulang aplikasi.

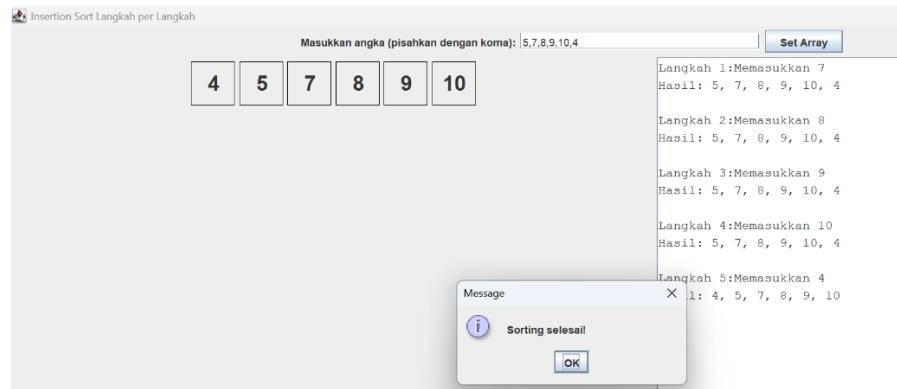
p. Method Main

```
175  
176 public static void main(String[] args) {
```

Method main merupakan titik awal jalannya program Java. Pada bagian ini dibuat objek InsertionSortGUI_1006 agar JFrame dapat ditampilkan ke layar. Program menggunakan `SwingUtilities.invokeLater()` agar GUI berjalan dengan aman pada thread Swing. Setelah program dijalankan, window GUI insertion sort akan muncul. Pengguna kemudian dapat memasukkan angka dan melihat proses sorting secara

visual. Method main menjadi bagian penting agar seluruh aplikasi dapat dijalankan dengan baik.

q. Hasil Output Program



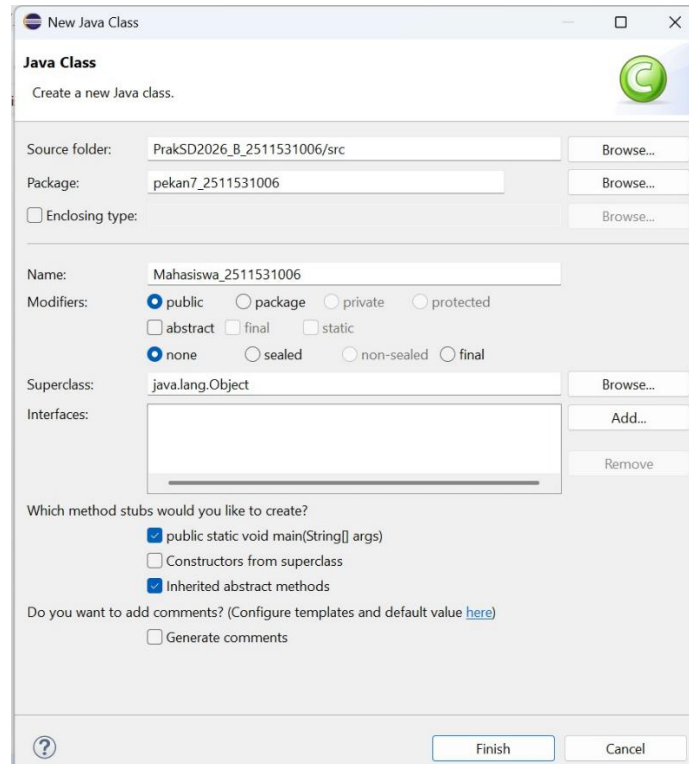
Gambar di atas menunjukkan hasil tampilan program GUI Insertion Sort yang telah berhasil dijalankan. Pada bagian input, pengguna memasukkan data array 5,7,8,9,10,4 ke dalam text field yang tersedia kemudian menekan tombol Set Array. Setelah itu program menampilkan array dalam bentuk visual menggunakan label kotak angka. Setiap tombol langkah dijalankan, program akan memproses insertion sort secara bertahap dan menampilkan hasil perubahan array pada area log di sebelah kanan. Tampilan ini membantu pengguna memahami proses pengurutan data secara lebih jelas dan interaktif.

Pada area langkah-langkah terlihat proses insertion sort dimulai dari memasukkan angka 7, 8, 9, 10, hingga angka 4. Saat angka 4 diproses, program melakukan pergeseran beberapa data karena nilai 4 lebih kecil dibanding angka sebelumnya. Setelah seluruh proses selesai, array berhasil terurut menjadi 4, 5, 7, 8, 9, 10. Program kemudian menampilkan pesan Sorting selesai! sebagai tanda bahwa proses pengurutan telah berhasil dilakukan tanpa error. Tampilan GUI ini membuat proses insertion sort lebih mudah dipahami dibanding program console biasa karena pengguna dapat melihat perubahan data secara langsung.

2.2 Langkah Langkah Tugas Praktikum

1. Program 1

a. Membuat Class Mahasiswa



Pada tahap ini dibuat sebuah class Java dengan nama Mahasiswa_2511531006. Class ini digunakan untuk merepresentasikan data mahasiswa dalam program. Pembuatan class dilakukan melalui menu New → Class pada Eclipse kemudian memasukkan nama class sesuai kebutuhan praktikum. Nama class ditambahkan NIM agar sesuai dengan format tugas yang diberikan dosen. Class ini nantinya digunakan untuk menyimpan atribut mahasiswa seperti nama, NIM, dan program studi. Dengan adanya class ini, data mahasiswa dapat dikelola dengan lebih terstruktur menggunakan konsep OOP (Object Oriented Programming).

b. Package Program

```
1 package pekan7_2511531006;  
2
```

Bagian package digunakan untuk mengelompokkan file program agar lebih rapi dan mudah dikelola. Semua class yang berkaitan dengan praktikum minggu ke-7 ditempatkan dalam package pekan7_2511531006. Dengan menggunakan package, program menjadi lebih terorganisir terutama jika jumlah class cukup banyak. Package juga membantu menghindari konflik nama antar class. Penamaan package biasanya disesuaikan dengan nama praktikum atau identitas mahasiswa. Pada program ini package menjadi tempat penyimpanan seluruh file Java yang digunakan.

c. Deklarasi Variabel atau Atribut

```
4 private String nama_2511531006;  
5 private String nim_2511531006;  
6 private String prodi_2511531006;  
7
```

Bagian ini digunakan untuk mendeklarasikan atribut yang dimiliki oleh class mahasiswa. Variabel nama_2511531006 digunakan untuk menyimpan nama mahasiswa. Variabel nim_2511531006 digunakan untuk menyimpan nomor induk mahasiswa, sedangkan prodi_2511531006 digunakan untuk menyimpan program studi mahasiswa. Semua atribut menggunakan access modifier private agar data tidak dapat diakses langsung dari luar class.

d. Constructor Mahasiswa

```
7  
8 public Mahasiswa_2511531006(String nama_2511531006, String nim_2511531006, String prodi_2511531006) {
```

Constructor digunakan untuk memberikan nilai awal pada objek saat objek dibuat. Pada constructor ini terdapat tiga parameter yaitu nama, NIM, dan program studi mahasiswa. Ketika objek mahasiswa dibuat, data yang dimasukkan akan langsung disimpan ke dalam atribut class. Constructor mempermudah proses inisialisasi data tanpa harus mengisi atribut satu per satu. Nama constructor harus sama dengan nama

class agar dapat dikenali oleh Java. Bagian ini merupakan salah satu konsep dasar penting dalam pemrograman berorientasi objek.

e. Pengisian Nilai Atribut

```
9      this.nama_2511531006 = nama_2511531006;  
10     this.nim_2511531006 = nim_2511531006;  
11     this.prodi_2511531006 = prodi_2511531006;  
12
```

Kode ini digunakan untuk mengisi atribut class dengan nilai parameter yang diterima constructor. Kata kunci `this` digunakan untuk membedakan antara atribut class dan parameter method yang memiliki nama sama. Nilai nama mahasiswa akan disimpan ke atribut `nama_2511531006`, begitu juga dengan NIM dan program studi. Dengan cara ini objek mahasiswa akan memiliki data masing-masing sesuai input yang diberikan. Penggunaan `this` sangat penting agar Java mengetahui variabel mana yang dimaksud. Teknik ini sering digunakan pada constructor di pemrograman Java.

f. Method Getter Nama

```
15 public String getNama_2511531006() {  
16     return nama_2511531006;  
17 }  
18
```

Method getter digunakan untuk mengambil atau menampilkan nilai atribut yang bersifat private. Method `getNama_2511531006()` digunakan untuk mengambil data nama mahasiswa. Karena atribut dibuat private, maka data tidak dapat diakses langsung dari luar class. Oleh sebab itu diperlukan getter agar data tetap bisa digunakan dengan aman. Method ini mengembalikan nilai berupa tipe data String. Getter merupakan bagian penting dalam konsep encapsulation pada Java.

g. Method Getter NIM

```
19 public String getNim_2511531006() {  
20     return nim_2511531006;  
21 }  
22
```

Method ini digunakan untuk mengambil nilai NIM mahasiswa yang tersimpan pada atribut class. Dengan menggunakan getter, data dapat diakses tanpa harus membuka akses langsung ke atribut private. Method akan mengembalikan nilai NIM dalam bentuk string. Getter membuat program menjadi lebih aman dan mudah dikontrol. Teknik ini juga membantu menjaga struktur program agar tetap rapi. Pada konsep OOP, getter sering dipasangkan dengan setter.

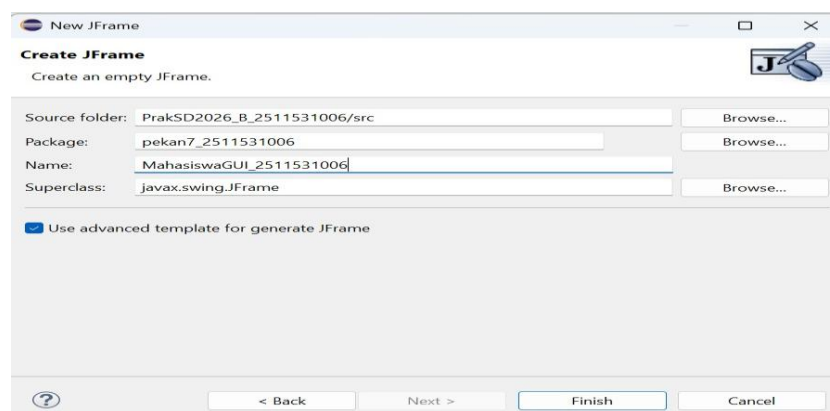
h. Method Getter Program Studi

```
22  
23  
24  
25  
26 }  
    public String getProdi_2511531006(){  
        return prodi_2511531006;  
    }
```

Method getter ini digunakan untuk mengambil data program studi mahasiswa. Nilai yang dikembalikan berasal dari atribut `prodi_2511531006`. Dengan adanya getter, data program studi dapat digunakan oleh class lain tanpa harus mengakses atribut secara langsung. Penggunaan getter membantu menjaga keamanan data pada program.

2. Program 2

a. Pembuatan JFrame MahasiswaGUI_2511531006



```

9
10 public class MahasiswaGUI_2511531006 extends JFrame {
11

```

Pada gambar pembuatan JFrame, dibuat sebuah form GUI baru dengan nama MahasiswaGUI_2511531006 pada package pekan7_2511531006. JFrame ini digunakan sebagai tampilan utama aplikasi untuk mengelola data mahasiswa dan melakukan proses sorting data menggunakan beberapa algoritma seperti Insertion Sort, Selection Sort, dan Bubble Sort. Dengan menggunakan JFrame, program menjadi lebih interaktif karena pengguna dapat memasukkan data melalui form, melihat tabel data mahasiswa, serta menjalankan proses sorting melalui tombol yang tersedia.

b. Import Library yang Digunakan

```

3 import javax.swing.*;
4
5 import javax.swing.table.DefaultTableModel;
6
7 import java.awt.*;
8 import java.awt.event.*;
9 import java.util.ArrayList;

```

Bagian import digunakan untuk memanggil library Java Swing, AWT, dan utilitas lainnya yang dibutuhkan dalam program. Library Swing digunakan untuk membuat komponen GUI seperti tombol, tabel, text field, dan combo box. Library java.awt digunakan untuk mengatur layout tampilan aplikasi. Selain itu, ArrayList digunakan untuk menyimpan data mahasiswa secara dinamis sehingga jumlah data dapat bertambah sesuai input pengguna. Dengan adanya import library ini, program dapat menjalankan fungsi tampilan GUI dan proses pengolahan data dengan baik.

c. Deklarasi Variabel dan Komponen GUI

```

13 private JTextField tfNama_2511531006;
14 private JTextField tfNim_2511531006;
15 private JTextField tfProdi_2511531006;
16 private JButton btnTambah_2511531006;
17 private JButton btnHapus_2511531006;
18 private JButton btnSort_2511531006;
19 private JComboBox<String> cbAlgoritma_2511531006;
20 private.JTable table_2511531006;
21 private DefaultTableModel tableModel_2511531006;
22 private JTextArea taLog_2511531006;
23 private ArrayList<Mahasiswa_2511531006> dataList_2511531006;

```

Pada bagian ini dilakukan deklarasi seluruh komponen yang digunakan dalam aplikasi GUI. Komponen tersebut terdiri dari JTextField untuk input data mahasiswa, JButton untuk tombol aksi, JTable untuk menampilkan data, JComboBox untuk memilih algoritma sorting, serta JTextArea untuk menampilkan proses sorting. Selain itu terdapat ArrayList<Mahasiswa_2511531006> yang berfungsi menyimpan objek mahasiswa. Deklarasi ini penting karena seluruh komponen akan digunakan pada berbagai method dalam program. Dengan adanya deklarasi variabel di awal class, pengelolaan data dan tampilan menjadi lebih mudah dan terstruktur.

d. Constructor MahasiswaGUI_2511531006

```
25 public MahasiswaGUI_2511531006() {  
26     setTitle("Aplikasi Sorting Nama Mahasiswa");  
27     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
28     setSize(900, 700);  
29     setLocationRelativeTo(null);  
30  
31     dataList_2511531006 = new ArrayList<>();  
32     initComponents_2511531006();  
33     setVisible(true);  
34 }  
25
```

Constructor digunakan untuk mengatur tampilan awal JFrame ketika program dijalankan. Pada bagian ini ditentukan judul aplikasi, ukuran window, posisi tampilan, serta operasi ketika tombol close ditekan. Selain itu dibuat juga ArrayList untuk menyimpan data mahasiswa dan dipanggil method initComponents_2511531006() untuk membangun seluruh tampilan GUI. Method setVisible(true) digunakan agar JFrame dapat langsung tampil ketika program dijalankan. Constructor ini menjadi pusat awal proses aplikasi sebelum pengguna mulai menginput data mahasiswa.

e. Method initComponents_2511531006()

```
35  
36 private void initComponents_2511531006() {
```

Method ini digunakan untuk membuat dan mengatur seluruh tampilan GUI aplikasi. Pada method ini dibuat panel utama, panel input data, tabel mahasiswa, combo box algoritma sorting, tombol sorting, serta area log proses sorting. Layout yang digunakan

adalah BorderLayout dan GridLayout agar tampilan menjadi lebih rapi. Selain itu pada bagian ini juga dilakukan penambahan event listener pada tombol tambah, hapus, dan sorting. Method ini sangat penting karena seluruh tampilan aplikasi dibangun di sini sehingga pengguna dapat berinteraksi langsung dengan program.

f. Pembuatan Panel Input Data Mahasiswa

```
39  
40      JPanel inputPanel_2511531006 = new JPanel(new GridLayout(4, 2, 5, 5));  
  
43      inputPanel_2511531006.add(new JLabel("Nama:"));  
44      tfNama_2511531006 = new JTextField();
```

Pada bagian ini dibuat panel input data mahasiswa menggunakan GridLayout. Panel ini digunakan untuk menampilkan form input nama, NIM, dan program studi mahasiswa. Selain itu terdapat tombol tambah data dan tombol hapus data. Dengan adanya panel input ini, pengguna dapat memasukkan data mahasiswa dengan mudah melalui tampilan GUI. Penggunaan GridLayout membuat posisi komponen menjadi lebih rapi dan tersusun secara teratur. JLabel digunakan untuk memberi keterangan input, sedangkan JTextField digunakan untuk memasukkan data mahasiswa.

g. Pembuatan JTable Mahasiswa

```
62  
63      String[] columnNames_2511531006 = {"No", "Nama", "NIM", "Prodi"};  
  
70  
71      table 2511531006 = new JTable(tableModel 2511531006);
```

Bagian ini digunakan untuk membuat tabel mahasiswa yang berfungsi menampilkan seluruh data mahasiswa yang telah diinput pengguna. Tabel memiliki beberapa kolom yaitu nomor, nama, NIM, dan program studi. Data yang tersimpan di dalam ArrayList nantinya akan ditampilkan pada tabel ini. Penggunaan JTable membuat tampilan data menjadi lebih rapi dan mudah dibaca pengguna. Selain itu tabel akan otomatis diperbarui ketika data ditambah, dihapus, atau diurutkan. Kolom tabel digunakan untuk menampilkan informasi mahasiswa secara terstruktur pada aplikasi GUI.

h. Pembuatan ComboBox Algoritma Sorting

```
78      cbAlgoritma_2511531006 = new JComboBox<>();
79      cbAlgoritma_2511531006.addItem("Insertion Sort");
80      cbAlgoritma_2511531006.addItem("Selection Sort");
81      cbAlgoritma_2511531006.addItem("Bubble Sort");
```

Pada bagian ini dibuat JComboBox yang digunakan untuk memilih algoritma sorting yang akan digunakan pengguna. Terdapat tiga pilihan algoritma yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Pengguna dapat memilih salah satu algoritma sebelum proses sorting dijalankan. Penggunaan combo box membuat aplikasi lebih fleksibel karena pengguna dapat mencoba beberapa algoritma sorting dalam satu program. Kode tersebut digunakan untuk menambahkan pilihan algoritma sorting ke dalam combo box.

i. Method tambahData_2511531006

```
118
119      private void tambahData_2511531006() {

128      dataList_2511531006.add(new Mahasiswa_2511531006(nama_2511531006, nim_2511531006, prodi_2511531006));
129
```

Method tambahData_2511531006() digunakan untuk menambahkan data mahasiswa ke dalam ArrayList. Program akan mengambil data dari text field berupa nama, NIM, dan program studi mahasiswa. Jika ada data yang kosong maka prakan menampilkan pesan peringatan menggunakan JOptionPane. Setelah data valid, program akan membuat objek mahasiswa baru lalu menyimpannya ke dalam ArrayList. Setelah itu tabel akan diperbarui menggunakan method updateTabel_2511531006() agar data langsung tampil pada GUI. Dan Method ini membantu pengguna memasukkan data mahasiswa secara dinamis ke dalam aplikasi.

j. Method hapusData_2511531006

```
136
137      private void hapusData_2511531006() {
138          int row_2511531006 = table_2511531006.getSelectedRow();
```



```
140         `dataList_2511531006`.remove(row_2511531006);
```

Method ini digunakan untuk menghapus data mahasiswa yang dipilih pengguna pada tabel. Program akan mengambil index baris yang dipilih menggunakan `getSelectedRow()`. Jika pengguna memilih salah satu data maka data t akan dihapus dari `ArrayList` menggunakan method `remove()`. Setelah data dihapus, tabel akan diperbarui kembali agar tampilan sesuai dengan data terbaru. Dengan adanya method ini pengguna dapat mengelola data mahasiswa dengan lebih mudah dan fleksibel. Method ini digunakan agar pengguna dapat menghapus data mahasiswa yang tidak diperlukan lagi.

k. Method `updateTabel_2511531006`

```
145 private void updateTabel_2511531006() {  
146     tableModel 2511531006.setRowCount(0);
```

```
149         tableModel_2511531006.addRow(new Object[]{  
150             no_2511531006++,  
151             m_2511531006.getNama_2511531006(),  
152             m_2511531006.getNim_2511531006(),  
153             m_2511531006.getProdi_2511531006()  
154         });  
155     }  
156 }
```

Method `updateTabel_2511531006()` digunakan untuk memperbarui isi tabel berdasarkan data terbaru yang ada pada `ArrayList`. Program akan menghapus seluruh isi tabel terlebih dahulu menggunakan `setRowCount(0)` kemudian memasukkan kembali seluruh data mahasiswa satu per satu ke tabel. Method ini dipanggil setiap kali data ditambah, dihapus, atau setelah proses sorting selesai dilakukan. Dengan adanya method ini, isi tabel akan selalu sesuai dengan data yang tersimpan di dalam program.

l. Method `getNamaList_2511531006`

```
157  
158 private String getNamaList_2511531006() {
```

Method `getNamaList_2511531006()` digunakan untuk mengambil seluruh nama mahasiswa dari `ArrayList` lalu mengubahnya menjadi bentuk string menggunakan `StringBuilder`. Method ini berfungsi untuk menampilkan urutan nama mahasiswa pada area log selama proses sorting berlangsung. Dengan adanya method ini, pengguna dapat melihat perubahan posisi data mahasiswa pada setiap langkah algoritma sorting. Penggunaan `StringBuilder` membuat proses penggabungan string menjadi lebih cepat dan efisien dibandingkan string biasa. Method ini membantu agar tampilan hasil sorting menjadi lebih rapi dan mudah dipahami pengguna.

m. Method `mulaiSorting_2511531006`

```
166  
167 private void mulaiSorting 2511531006() {
```

Method `mulaiSorting_2511531006()` digunakan untuk menjalankan proses sorting sesuai algoritma yang dipilih pengguna pada combo box. Program akan mengecek apakah data mahasiswa kosong atau tidak sebelum sorting dilakukan. Setelah itu program membaca pilihan algoritma seperti Insertion Sort, Selection Sort, atau Bubble Sort lalu memanggil method sorting yang sesuai. Hasil proses sorting akan ditampilkan pada area log dan tabel diperbarui kembali agar sesuai dengan data terbaru. Method ini menjadi pusat utama proses pengurutan data mahasiswa pada aplikasi GUI.

n. Method `insertionSort_2511531006`

```
188  
189 private void insertionSort_2511531006() {
```

Method `insertionSort_2511531006()` digunakan untuk mengurutkan data mahasiswa menggunakan algoritma Insertion Sort. Algoritma ini bekerja dengan mengambil satu data kemudian membandingkannya dengan data sebelumnya sampai posisi yang sesuai ditemukan. Pengurutan dilakukan berdasarkan nama mahasiswa menggunakan `compareToIgnoreCase()` sehingga huruf besar dan kecil tidak mempengaruhi hasil sorting. Setiap langkah proses sorting akan ditampilkan pada area log agar pengguna dapat memahami jalannya algoritma secara bertahap. Insertion Sort cocok digunakan

untuk jumlah data yang tidak terlalu banyak karena algoritmanya sederhana dan mudah dipahami.

o. Method selectionSort_2511531006

```
203  
204 private void selectionSort_2511531006() {
```

Method selectionSort_2511531006() digunakan untuk mengurutkan data mahasiswa menggunakan algoritma Selection Sort. Program akan mencari nama mahasiswa dengan urutan alfabet paling kecil kemudian menukarnya dengan posisi data saat ini. Proses tersebut dilakukan berulang hingga seluruh data berhasil terurut dengan benar. Setiap langkah sorting juga ditampilkan pada area log sehingga pengguna dapat melihat proses pengurutan secara bertahap. Algoritma Selection Sort bekerja dengan cara memilih elemen terkecil pada setiap iterasi proses sorting.

p. Method bubbleSort_2511531006

```
222  
223 private void bubbleSort_2511531006() {
```

Method bubbleSort_2511531006() digunakan untuk mengurutkan data mahasiswa menggunakan algoritma Bubble Sort. Algoritma ini bekerja dengan membandingkan dua data yang berdekatan kemudian menukarnya jika posisi data masih salah. Proses dilakukan terus menerus hingga seluruh data terurut berdasarkan nama mahasiswa. Pada program ini proses perbandingan menggunakan compareToIgnoreCase() agar huruf besar dan kecil tidak mempengaruhi hasil sorting. Bubble Sort merupakan algoritma sederhana yang mudah dipahami namun kurang efisien untuk data dalam jumlah besar.

q. Penggunaan compareToIgnoreCase()

```
228  
        .compareToIgnoreCase(`
```

Method `compareToIgnoreCase()` digunakan untuk membandingkan dua data string tanpa memperhatikan huruf besar dan kecil. Pada program ini method tersebut digunakan saat proses sorting nama mahasiswa berlangsung. Jika hasil perbandingan bernilai lebih besar dari nol maka posisi data akan ditukar sesuai algoritma sorting yang digunakan. Penggunaan method ini membuat hasil pengurutan menjadi lebih konsisten karena nama seperti “Andi” dan “andi” dianggap sama. Method ini sangat penting dalam proses sorting data bertipe string pada Java.

r. Penggunaan ArrayList

```
23     private ArrayList<Mahasiswa_2511531006> dataList_2511531006;  
24
```

ArrayList digunakan sebagai tempat penyimpanan data mahasiswa secara dinamis pada program. Berbeda dengan array biasa, ArrayList dapat menambah dan menghapus data tanpa harus menentukan ukuran sejak awal. Pada program ini setiap data mahasiswa yang diinput pengguna akan disimpan ke dalam `dataList_2511531006`. Data tersebut kemudian digunakan untuk ditampilkan pada tabel maupun diproses pada algoritma sorting. Penggunaan ArrayList membuat program menjadi lebih fleksibel dan mudah dikembangkan.

s. Penggunaan JOptionPane

```
125     JOptionPane.showMessageDialog
```

JOptionPane digunakan untuk menampilkan pesan dialog kepada pengguna seperti pesan error, peringatan, atau informasi tertentu. Pada program ini JOptionPane digunakan ketika pengguna belum mengisi seluruh data input atau ketika data kosong saat proses sorting dilakukan. Dengan adanya pesan dialog, pengguna dapat mengetahui kesalahan yang terjadi secara langsung melalui tampilan pop-up. Penggunaan JOptionPane membuat aplikasi menjadi lebih user friendly dan interaktif. Selain itu pengguna menjadi lebih mudah memahami kondisi program saat dijalankan.

t. Method main()

```

237
238 public static void main(String[] args_2511531006) {
239     new MahasiswaGUI_2511531006();
240 }
241 }
242

```

Method main() merupakan titik awal program dijalankan pada Java. Pada bagian ini dibuat objek baru dari class MahasiswaGUI_2511531006 sehingga constructor otomatis dipanggil dan seluruh tampilan GUI muncul di layar. Method ini sangat penting karena tanpa method main(), aplikasi Java tidak dapat dijalankan. Setelah program dijalankan, pengguna dapat langsung melakukan input data mahasiswa dan menjalankan proses sorting. Method main() menjadi awal eksekusi seluruh program aplikasi GUI sorting data mahasiswa.

u. Hasil Output Program

```

=== Insertion Sort ===
Data Awal: [sasa, winda, quenza]

Langkah 1: [sasa, winda, quenza]
Langkah 2: [quenza, sasa, winda]
Hasil: [quenza, sasa, winda]

```

Pada hasil program Insertion Sort terlihat bahwa data awal yang dimasukkan adalah [sasa, winda, quenza]. Proses sorting dilakukan dengan membandingkan data satu per satu lalu memindahkan posisi data sesuai urutan alfabet. Setelah proses selesai, data berhasil diurutkan menjadi [quenza, sasa, winda]. Hal ini menunjukkan bahwa algoritma Insertion Sort dapat mengurutkan data nama secara ascending dengan baik.

```

=== Selection Sort ===
Data Awal: [quenza, sasa, winda]

Pass 1: [quenza, sasa, winda]
Pass 2: [quenza, sasa, winda]
Hasil: [quenza, sasa, winda]

```

Pada hasil program Selection Sort terlihat data awal yaitu [quenza, sasa, winda]. Algoritma bekerja dengan mencari data terkecil pada setiap proses lalu menukarkannya ke posisi depan. Karena data sudah terurut dari awal, maka pada setiap pass tidak terjadi perubahan posisi data. Hasil akhir tetap [quenza, sasa, winda] sehingga proses sorting berjalan dengan benar.

```
=== Bubble Sort ===  
Data Awal: [kania, quenza, sasa]  
  
Pass 1: [kania, quenza, sasa]  
Pass 2: [kania, quenza, sasa]  
Hasil: [kania, quenza, sasa]
```

Pada hasil program Bubble Sort terlihat data awal yaitu [kania, quenza, sasa]. Algoritma Bubble Sort bekerja dengan membandingkan dua data yang berdekatan secara berulang. Karena data sudah berada dalam urutan alfabet, maka tidak terjadi pertukaran data pada setiap pass. Hasil akhir tetap [kania, quenza, sasa] dan menunjukkan bahwa program Bubble Sort berhasil dijalankan dengan baik.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting sangat penting dalam proses pengolahan data karena dapat mengurutkan data menjadi lebih rapi dan terstruktur. Pada praktikum ini telah berhasil dibuat program Insertion Sort, Selection Sort, dan Bubble Sort menggunakan bahasa pemrograman Java baik berbasis console maupun GUI. Setiap algoritma memiliki cara kerja yang berbeda dalam melakukan proses pengurutan data. Pembuatan aplikasi GUI menggunakan Java Swing membuat program menjadi lebih interaktif dan mudah digunakan pengguna. Penggunaan komponen seperti JFrame, JTable, JTextField, JButton, dan JOptionPane membantu dalam proses input, output, dan pengolahan data mahasiswa. Selain itu, mahasiswa juga memahami penggunaan ArrayList untuk menyimpan data object mahasiswa secara dinamis. Melalui praktikum ini mahasiswa dapat memahami konsep struktur data, algoritma sorting, dan pemrograman berorientasi objek dengan lebih baik. Praktikum ini juga melatih kemampuan analisis logika pemrograman serta meningkatkan pemahaman dalam membuat aplikasi sederhana menggunakan Java.

DAFTAR PUSTAKA

Deitel, P., & Deitel, H. *Java How to Program*. Pearson Education, 2018.

Oracle Academy. *Java Foundations Student Guide*. Oracle Corporation, 2021.

Sedgewick, R., & Wayne, K. *Algorithms Fourth Edition*. Addison-Wesley, 2011.